

Práctica 1. RRNN y Cálculo Paralelo

Siria Catherine Iñiguez Brito

12 de abril de 2026

1. Ejemplo 1: Multiplicación Matriz-Vector

Se evalúa la operación $\mathbf{b} = \mathbf{A}\mathbf{u}$ para una matriz de dimensiones considerables, comparando el método directo de MATLAB frente a implementaciones iterativas.

Cuadro 1: Tiempos de ejecución y Speedup (Ejemplo 1)

Método	Tiempo (s)	Speedup (vs Directo)	Speedup (vs Sec.)
Directo	26.17	—	—
Secuencial (<i>for</i>)	90.72	0.288x	—
Paralelo (<i>parfor</i>)	141.25	0.185x	0.642x

El resultado muestra un *speedup* inferior a la unidad en la versión paralela. Esto se explica porque la operación matricial directa en MATLAB ya utiliza librerías optimizadas. Al forzar un bucle `parfor`, el *overhead* introducido por la comunicación en el proceso con los *workers* supera cualquier ganancia computacional.

2. Ejemplo 2: Cálculo de π mediante Montecarlo

Se aproxima el valor de π mediante una simulación estocástica de 10^8 y $2 \cdot 10^8$ iteraciones.

Los resultados muestran que el *speedup* alcanzado para $n = 10^8$ es de 2,2, mientras que para $n = 2 \cdot 10^8$ se sitúa en 2,0.

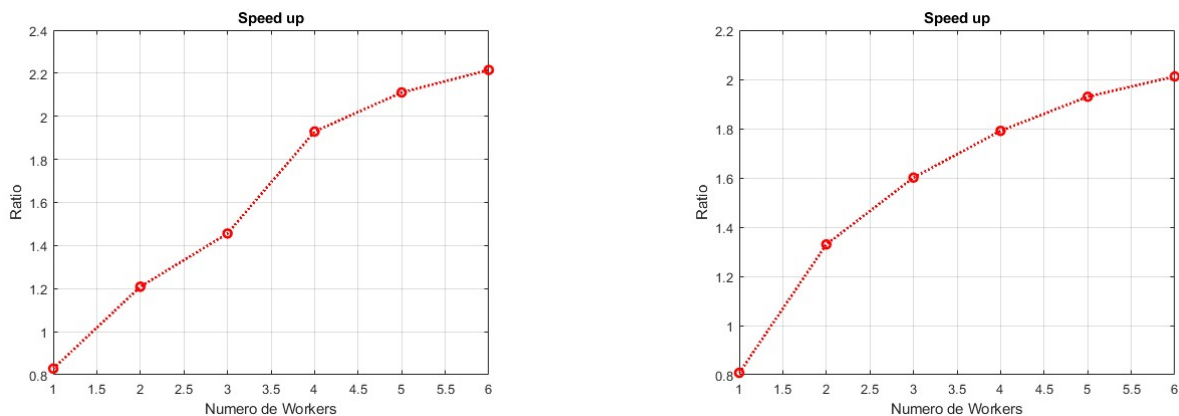


Figura 1: Curvas de Speedup para distintas cargas de trabajo (n).

La ligera disminución del Speedup al doblar la carga ($n = 2 \cdot 10^8$) sugiere que el sistema alcanza un cuello de botella en la gestión de memoria o en la generación masiva de números aleatorios.

3. Ejercicio 1: Conversión ASCII

Se implementa la transformación de una cadena de texto a su representación numérica ASCII.

Cuadro 2: Rendimiento en la conversión de $1,2 \cdot 10^6$ caracteres

Método	Tiempo (s)	Speedup
Secuencial	3.7435	—
SPMD	1.2403	3.01x
PARFOR	1.0141	3.69x

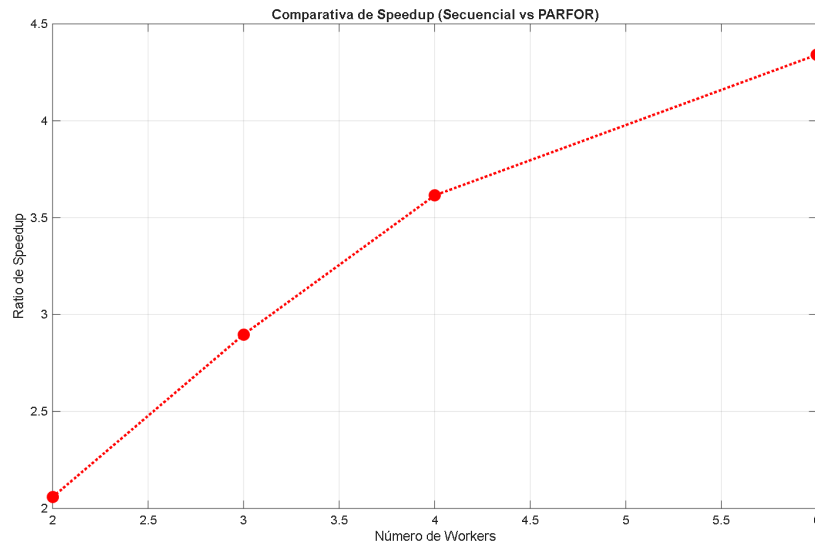


Figura 2: Evolución del Speedup con 2, 3, 4 y 6 workers.

A diferencia del Ejemplo 1, esta tarea no está vectorizada por defecto de forma tan eficiente, lo que permite que el paralelismo explícito brinde una ganancia real de tiempo.